Project Title : Database Migration and Management: Transitioning from MySQL to MongoDB for Legacy Application

Team Members:

1. Mohammed Mafaz — CAN ID : RM20240005431
2. Mohammed Suhail Aliakbara — CAN ID : RM20240005456
3. Syed Abdul Muqsit — CAN ID : RM20240005454
4. Arsh Umar Hussain Umar — CAN ID : RM20240005423

Institution Name : Anjuman Institute Of Technology And Management, Bhatkal

---

Phase 3 - Implementation of Project

1. Objective

- The goal of this phase is to implement the core components of the Library Management System (LMS) while successfully migrating legacy data from MySQL to MongoDB. This phase focuses on:
  - Migrating and validating legacy data using a custom migration script.
  - Developing a robust back-end using Node.js, Express, and Mongoose.
  - Creating a React-based front-end for system administration with Tailwind CSS.
  - Verifying all API endpoints using Postman for thorough testing.

---

2. Back-End Development

Overview

The back-end has been re-architected to interact with MongoDB using Mongoose, replacing the legacy Sequelize-based MySQL implementation. The models (for books, publishers, authors, copies, lending, library branches, and cards) have been defined and refined. A custom migration script (migrateData.js) transferred data from MySQL to MongoDB while preserving relationships using ObjectId references.

Implementation Details

- **Database Integration:**
  MongoDB is now the primary data store. Mongoose schemas have been implemented for all entities, with appropriate validation and reference mappings.
- **API Endpoints:**
  RESTful endpoints have been developed for:
  - **Publishers:** POST /api/publishers, GET /api/publishers
  - **Books:** CRUD endpoints for managing books (with fields such as title, ISBN, and publisher reference).
  - **Authors, Book Copies, Lending, Library Branches, and Cards:** Similar endpoints have been implemented.
- **Data Relationships:**
  Relationships are maintained using MongoDB's ObjectId references. For instance, each book document references its publisher and includes an array of author IDs mapped from the legacy relational data.
- **Models:**
  The back-end models are structured to represent the necessary fields and relationships. Each model follows Mongoose best practices and has been tested individually.
- **Testing:**
  All API endpoints have been verified using Postman. Sample requests (with JSON payloads) have been executed to ensure data creation, retrieval, updating, and deletion work as expected.

---

3. Front-End Development

Overview

The administration panel for the LMS has been built using React.js. This front-end provides a user-friendly interface for managing the system, including tasks such as adding or updating publishers, books, and tracking lending activities.

Implementation Details

- **Component Structure:**
  The React application (see uploaded source files) includes:
  - A dashboard for overall system metrics.
  - Dedicated modules for managing publishers, books, authors, and lending records.
  - Forms and data tables that allow administrators to perform CRUD operations.

- **State Management and API Integration:**
  The front-end leverages either the Context API or Redux for managing state. Axios is used to interact with the back-end RESTful APIs.
- **User Interface:**
  The UI is styled using Tailwind CSS to ensure the application is responsive and accessible on various devices.
- **Testing:**
  While the primary API testing has been done via Postman, the front-end has been manually tested to verify data display and form functionality.

---

4. Integration and Deployment

Integration

- The React front-end communicates with the Node.js/Express back-end through well-defined RESTful API endpoints.
- The back-end interfaces with MongoDB using Mongoose, with all models and data relationships validated.
- End-to-end testing has been performed using Postman to ensure seamless data flow and API functionality.

Deployment

- The application is currently running on a local development server.
- Future deployment plans include hosting the back-end on a cloud platform (e.g., IBM Cloud or similar) and configuring the front-end for production using build tools like Webpack or Create React App.

---

5. Testing and Quality Assurance

API Testing

- **Tool Used:**
  All API endpoints have been tested using Postman.
- **Process:**
  Each endpoint was validated by sending HTTP requests with appropriate JSON payloads

and verifying the responses. This ensured that CRUD operations on entities like publishers, books, and authors function correctly.

- **Outcome:**
  Testing confirmed that data is correctly created, retrieved, updated, and deleted in the MongoDB database, and that relationships between entities are maintained as expected.

Front-End Verification

- The React administration panel was tested by interacting with all features (form submissions, data displays, updates, etc.) to ensure a seamless user experience.
- Manual verification confirmed that the front-end correctly reflects the data managed via the back-end.

---

6. Challenges and Future Enhancements

Challenges Encountered

- **Data Mapping:**
  Transforming relational data into a document-oriented model required careful handling of relationships and ObjectId mappings.
- **API Refactoring:**
  Rewriting SQL-based logic to work with MongoDB and Mongoose was a significant effort.
- **Front-End Integration:**
  Ensuring that the React front-end adapted to the new API endpoints and data structure was an iterative process.

Future Enhancements

- **Enhanced Reporting and Analytics:**
  Develop advanced dashboards and reporting features for better system insights.
- **User Authentication:**
  Implement JWT-based authentication for secure access to the administration panel.
- **Performance Optimization:**
  Further optimize back-end queries and introduce caching strategies.
- **Mobile Compatibility:**
  Improve the responsiveness and usability of the administration panel on mobile devices.

7. Conclusion

This implementation phase has successfully integrated the migrated MongoDB database with a Node.js/Express back-end and a React-based administration panel. All API endpoints have been thoroughly tested using Postman, and the system's data integrity and functionality have been validated. The project is now well-positioned for future enhancements and scaling.

Screenshots of Code and Front-end

```
// migrateData.js
const { MongoClient, ObjectId } = require('mongodb');
const mysql = require('mysql2');

// MongoDB connection details
const mongoUri = 'mongodb://localhost:27017';
const mongoDbName = 'library_mongo';

// MySQL connection details
const mysqlConfig = {
  host: 'localhost',
  user: 'root',
  password: '',
  database: 'library',
};

const mysqlConnection = mysql.createConnection(mysqlConfig).promise();

async function migrateData() {
  let mongoClient;
  try {
    // Connect to MongoDB
    mongoClient = await MongoClient.connect(mongoUri);
    console.log('Connected to MongoDB');
    const db = mongoClient.db(mongoDbName);

    console.log('Starting data migration...');

    // Fetch data from MySQL
    const [publishers] = await mysqlConnection.query('SELECT * FROM
publishers');
    const [authors] = await mysqlConnection.query('SELECT * FROM authors');
```

```
    const [books] = await mysqlConnection.query('SELECT * FROM books');
    const [bookCopies] = await mysqlConnection.query('SELECT * FROM
book_copies');
    const [bookLending] = await mysqlConnection.query('SELECT * FROM
book_lending');
    const [libraryBranches] = await mysqlConnection.query('SELECT * FROM
library_branches');
    const [cards] = await mysqlConnection.query('SELECT * FROM cards');
    const [bookAuthorAssociations] = await mysqlConnection.query('SELECT *
FROM book_authors');

    // 1. Insert Publishers & Create Mapping
    const publisherMap = {};
    const mongoPublishers = publishers.map(pub => {
      const mongoId = new ObjectId();
      publisherMap[pub.id] = mongoId;
      return { _id: mongoId, name: pub.name };
    });
    await db.collection('publishers').insertMany(mongoPublishers);

    // 2. Insert Authors & Create Mapping
    const authorMap = {};
    const mongoAuthors = authors.map(auth => {
      const mongoId = new ObjectId();
      authorMap[auth.id] = mongoId;
      return { _id: mongoId, name: auth.name };
    });
    await db.collection('authors').insertMany(mongoAuthors);

    // 3. Insert Books (Referencing Authors & Publishers)
    const bookMap = {};
    const bookInserts = books.map(book => {
      const mongoId = new ObjectId();
      bookMap[book.id] = mongoId;
      return {
        _id: mongoId,
        title: book.title,
        isbn: book.isbn,
        publisherId: publisherMap[book.publisherId] || null,
        authors: bookAuthorAssociations
          .filter(assoc => assoc.bookId === book.id)
          .map(assoc => authorMap[assoc.authorId])
      };
```

```
});
await db.collection('books').insertMany(bookInserts);

// 4. Insert Library Branches & Create Mapping
const branchMap = {};
const mongoBranches = libraryBranches.map(branch => {
  const mongoId = new ObjectId();
  branchMap[branch.id] = mongoId;
  return { _id: mongoId, location: branch.location };
});
await db.collection('library_branches').insertMany(mongoBranches);

// 5. Insert Book Copies
const bookCopiesMap = {};
const bookCopiesInserts = bookCopies.map(copy => {
  const mongoId = new ObjectId();
  bookCopiesMap[copy.id] = mongoId;
  return {
    _id: mongoId,
    bookId: bookMap[copy.bookId],
    branchId: branchMap[copy.libraryBranchId],
    availableCopies: copy.availableCopies,
    totalCopies: copy.totalCopies
  };
});
await db.collection('book_copies').insertMany(bookCopiesInserts);

// 6. Insert Cards & Create Mapping
const cardMap = {};
const mongoCards = cards.map(card => {
  const mongoId = new ObjectId();
  cardMap[card.id] = mongoId;
  return { _id: mongoId, holderName: card.holderName };
});
await db.collection('cards').insertMany(mongoCards);

// 7. Insert Book Lending (Referencing Book Copies & Cards)
const bookLendingInserts = bookLending.map(lend => ({
  bookCopyId: bookCopiesMap[lend.bookCopyId],
  cardId: cardMap[lend.cardId],
  borrowDate: lend.borrowDate,
  returnDate: lend.returnDate || null,
}));
```

```
    await db.collection('book_lending').insertMany(bookLendingInserts);

    console.log('Data migration completed successfully!');
  } catch (error) {
    console.error('Error during data migration:', error);
  } finally {
    if (mongoClient) await mongoClient.close();
    await mysqlConnection.end();
  }
}

migrateData();
```

Back-End Api Endpoints :

Post : http://localhost:3000/api/books

```
{
    "title":"Book Title",
    "isbn" :"isbnNumber1",
    "publisherId":"67bd964790acf237f4fa33e3",
    "authors":["67bd964790acf237f4fa33e7","67bd964790acf237f4fa33e8"],
    "copies" :[
        {
            "libraryBranchId" : "67bd964790acf237f4fa33f0",
            "avialableCopies" : 50,
            "totalCopies" : 50
        }
    ]
}
```

Get : http://localhost:3000/api/books

```
[
    {
        "_id": "67bd964790acf237f4fa33ea",
        "title": "Harry Potter and the Sorcerer's Stone",
        "isbn": "9780747532743",
        "publisherId": "67bd964790acf237f4fa33e2",
        "authors": [
            "67bd964790acf237f4fa33e6" ]  },
```

```
    {
        "_id": "67bd964790acf237f4fa33eb",
        "title": "Book Title",
        "isbn": "isbnNumber1",
        "publisherId": "67bd964790acf237f4fa33e3",
        "authors": [
            "67bd964790acf237f4fa33e7",
            "67bd964790acf237f4fa33e8"
        ]
    },
```


Mongoose Model example :

Book model :

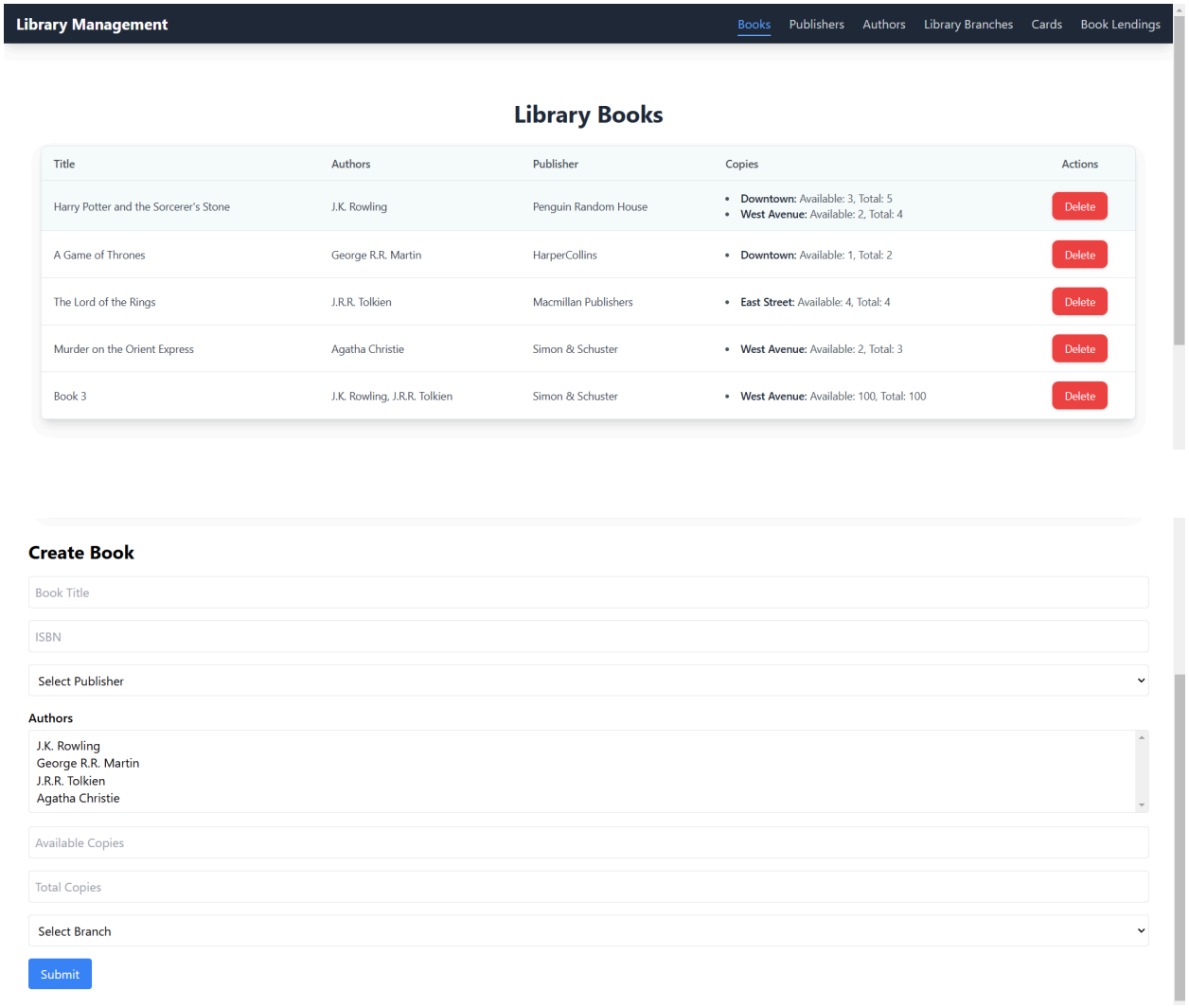
```
const mongoose = require('mongoose');

const bookSchema = new mongoose.Schema({
   title: { type: String, required: true },
   isbn: { type: String, required: true },
   publisher: { type: mongoose.Schema.Types.ObjectId, ref: 'Publisher', required: true },
   authors: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Author', required: true }],
   copies: [{
      bookId : {type: mongoose.Schema.Types.ObjectId},
      libraryBranchId: { type: Number },
      availableCopies: { type: Number },
      totalCopies: { type: Number },
   }],
});

const Book = mongoose.model('Book', bookSchema);

module.exports = Book;
```

Front-End :


Library Management
Books
Publishers
Authors
Library Branches
Cards
Book Lendings
Library Books
Title
Authors
Publisher
Copies
Actions
Harry Potter and the Sorcerer's Stone
J.K. Rowling
Penguin Random House
Downtown: Available: 3, Total: 5
West Avenue: Available: 2, Total: 4
Delete
A Game of Thrones
George R.R. Martin
HarperCollins
Downtown: Available: 1, Total: 2
Delete
The Lord of the Rings
J.R.R. Tolkien
Macmillan Publishers
East Street: Available: 4, Total: 4
Delete
Murder on the Orient Express
Agatha Christie
Simon & Schuster
West Avenue: Available: 2, Total: 3
Delete
Book 3
J.K. Rowling, J.R.R. Tolkien
Simon & Schuster
West Avenue: Available: 100, Total: 100
Delete

Create Book
Book Title
ISBN
Select Publisher
Authors
J.K. Rowling
George R.R. Martin
J.R.R. Tolkien
Agatha Christie
Available Copies
Total Copies
Select Branch
Submit